Trust-FedAvg: A Trust-Aware Federated Learning Benchmarking Framework Extending FedBench with Multi-Dataset, Non-IID, and Communication-Overhead Evaluation

CUJ Interns – Amrita Group
Central University of Jharkhand (CUJ)
Research Internship, National Institute of Technology Jamshedpur

***Abstract*—Federated Learning (FL) enables collaborative model training across distributed clients without centralizing raw data, but practical deployments must be evaluated not only on model accuracy but also on robustness to data heterogeneity, communication cost, and system-level resource usage. This report presents *Trust-FedAvg*, a trust-aware aggregation strategy implemented within an extended FedBench-style benchmarking pipeline, and evaluates it against standard FedAvg across three image classification datasets – MNIST, FashionMNIST, and CIFAR-10 – under both Independent and Identically Distributed (IID) and Non-IID (Dirichlet, $\alpha = 0.5$) client data partitions. The framework further integrates communication-overhead estimation and live network monitoring via `psutil`, in line with the system-level Key Performance Indicator (KPI) philosophy proposed by the FedBench platform. Across twelve experimental configurations (3 datasets $\times$ 2 distributions $\times$ 2 aggregation strategies), Trust-FedAvg matches or exceeds FedAvg accuracy in 5 of 6 IID/Non-IID dataset pairs, with the largest gain observed on FashionMNIST under Non-IID partitioning (+0.99 percentage points) and the smallest accuracy degradation under data heterogeneity, confirming that trust-weighted aggregation partially compensates for client data skew. Results are discussed in relation to the benchmarking methodology proposed by Errico et al. for cloud-edge-IoT FL deployments.**

***Index Terms*—Federated Learning, Trust-Aware Aggregation, FedAvg, Non-IID Data, Benchmarking, Communication Overhead, FedBench**

I. Introduction

Federated Learning (FL) allows multiple clients to collaboratively train a shared global model while keeping raw data local, exchanging only model parameters or gradients with a coordinating server [1]. This paradigm is attractive whenever data is privacy-sensitive or naturally distributed across edge devices. However, two practical challenges complicate real-world FL deployments: (i) client data is rarely Independent and Identically Distributed (IID), and statistical heterogeneity across clients can degrade convergence and final accuracy; and (ii) not all client updates are equally reliable, as clients may have noisy data, limited samples, or inconsistent training behavior across rounds.

Recent benchmarking efforts have emphasized that FL systems should be evaluated using standardized, reproducible procedures that go beyond accuracy alone, incorporating system-level metrics such as communication cost and resource utilization [1]. Motivated by this perspective, this work extends a FedBench-style experimental pipeline along three axes:

- **Multi-dataset evaluation**: MNIST is added alongside FashionMNIST and CIFAR-10 to assess generalization across datasets of increasing visual complexity.
- **Non-IID partitioning**: client data is partitioned using a Dirichlet distribution ($\alpha = 0.5$) to simulate realistic data heterogeneity, in addition to standard IID partitioning.
- **Trust-aware aggregation**: a Trust-FedAvg strategy is proposed, in which each client's contribution to the global model is weighted by a dynamically computed trust score, in addition to its local dataset size.

We additionally instrument the pipeline with a network-traffic monitor that estimates per-round communication overhead from model parameter counts and records live system-wide network counters, mirroring the system-KPI methodology adopted by FedBench for cloud-edge-IoT deployments [1].

Statistical heterogeneity is widely recognized as one of the central obstacles to reliable FL deployment. When client data distributions diverge significantly from the population distribution – for instance, when a client observes only a subset of class labels – the local objective optimized by each client can diverge from the true global objective, a phenomenon often referred to as *client drift*. Standard FedAvg, which weights client contributions purely by local dataset size, has no mechanism to distinguish a client whose update is reliable but numerically small in influence from a client whose update is large in influence but poorly aligned with the global objective due to skewed local data. This motivates the introduction of an additional weighting signal, grounded in observed training behavior, that can complement size-based weighting.

The contributions of this report can be summarized as follows:

1) We design and implement a lightweight, loss-based trust score that combines an instantaneous inter-client quality term with an exponentially-smoothed historical consistency term, requiring no additional communication

beyond the standard FedAvg protocol (loss values are already reported by clients during evaluation).
2) We integrate this trust score into a Trust-FedAvg aggregation rule and benchmark it against standard FedAvg across three datasets of increasing visual complexity (MNIST, FashionMNIST, CIFAR-10) and two client data-distribution regimes (IID and Dirichlet Non-IID, $\alpha = 0.5$).
3) We extend the experimental pipeline with system-level instrumentation – per-round communication-cost estimation and live OS-level network monitoring – aligned with the benchmarking philosophy proposed by FedBench [1], enabling joint accuracy/communication-cost analysis.
4) We provide a detailed, per-dataset discussion of when and why trust-aware aggregation helps, and identify the conditions (task complexity, training budget) under which its benefit diminishes, informing future extensions of the framework.

The remainder of this report is organized as follows. Section II reviews related FL benchmarking platforms and robust/trust-aware aggregation strategies. Section III details the system design, datasets, partitioning strategy, trust-score formulation, and aggregation rules. Section IV describes the experimental setup. Section V presents quantitative results across all twelve configurations. Section VI discusses the implications of these results and compares the adopted benchmarking methodology to that of FedBench. Section VII discusses limitations and threats to validity, and Section VIII concludes with directions for future work.

II. Related Work

FedBench [1] is an extensible platform for deploying and benchmarking FL algorithms over heterogeneous cloud-edge-IoT infrastructures. It aligns its measurement model with the REDRAW framework, collecting both model-level KPIs (accuracy, loss, convergence) and system-level KPIs (CPU utilization, RAM usage, disk I/O, and network traffic via tools such as `nmon`, `pidstat`, `perf`, and `tshark`). FedBench demonstrates its platform on a Fashion-MNIST baseline and a sleep-apnea detection case study using a heterogeneous testbed of a PC, a Raspberry Pi 400, and a smartphone, reporting that CPU usage remained below 33% and RAM usage below 50% of device capacity across devices, while federated accuracy on the apnea task (0.88/0.87/0.86 across the three client types) closely tracked centralized training accuracy (0.84) at the cost of additional communication rounds.

Unlike FedBench, which focuses on deployment automation and real heterogeneous-device orchestration (Kubernetes, Helm, ONNX-based proxies), the present work operates at the algorithmic and simulation level, focusing on the interaction between client data heterogeneity and trust-weighted aggregation. The two efforts are complementary: the benchmarking philosophy (standardized KPIs, IID/Non-IID comparison, communication-overhead accounting) adopted here follows the same system-aware evaluation principles proposed by FedBench, while the trust-aware aggregation strategy addresses a robustness dimension not explored in the original FedBench case studies.

Trust-aware and robust aggregation strategies have also been studied broadly in the FL literature as mechanisms to mitigate the effect of unreliable, noisy, or adversarial clients, typically by reweighting client updates according to a quality or consistency score rather than relying solely on dataset-size weighting, as in standard FedAvg. The original FedAvg algorithm [2] establishes the size-weighted averaging baseline used throughout this work, while broader surveys of the field [3] catalogue robustness, personalization, and communication-efficiency as three of the principal open problems in production FL systems. Benchmark suites such as LEAF [4] and FedScale [5] provide standardized datasets and unified evaluation protocols for comparing FL algorithms at scale, but, like most algorithm-centric frameworks, they do not natively integrate deployment automation or live system-level monitoring of communication and resource usage – the gap that FedBench specifically targets through its proxy-based, Kubernetes/Helm-orchestrated architecture and its ONNX-based model interoperability layer for heterogeneous edge devices [1].

Robust aggregation rules in the broader literature typically fall into one of three categories: (i) statistical outlier rejection methods, such as Krum [13] and coordinate-wise trimmed-mean aggregation [14], which discard or down-weight updates that deviate substantially from the geometric median or trimmed-mean of all client updates; (ii) reputation or trust-score-based methods, which maintain a persistent, slowly-evolving estimate of each client's reliability across rounds and use it to reweight contributions, as adopted in this work and as explored in incentive-aware FL settings that combine reputation scores with contract-theoretic client selection [15]; and (iii) validation-based methods, which evaluate each client's update against a held-out server-side validation set before incorporating it into the global model. The Trust-FedAvg strategy proposed here belongs to the second category: it requires no additional held-out data at the server and adds negligible computational overhead, since the loss values used to compute trust scores are already produced as a byproduct of local training.

Beyond robustness, a parallel line of work addresses statistical heterogeneity directly through modified local-update or aggregation rules rather than through reweighting alone. FedProx [9] adds a proximal regularization term to each client's local objective to limit divergence from the global model under heterogeneous and partially-completed local updates, while SCAFFOLD [10] introduces control variates to correct for client drift directly in the local gradient updates, achieving faster convergence under Non-IID conditions at the cost of additional per-client state that must be communicated alongside model weights. Empirical studies have further quantified the magnitude of accuracy degradation introduced by non-identical client data distributions in visual classification tasks [11], [12], providing the empirical motivation for the IID-

versus-Non-IID comparison methodology adopted in Section V of this report. Unlike FedProx and SCAFFOLD, which modify the local optimization procedure itself, Trust-FedAvg operates purely at the aggregation stage and is therefore compatible with unmodified local training code, simplifying integration into existing FedAvg-based pipelines such as the one extended here.

The broader applicability of FL to resource-constrained IoT and edge deployments, and the systems-level challenges this introduces – including intermittent connectivity, heterogeneous compute capability, and the need for production-grade orchestration – has been surveyed extensively [16], [17], reinforcing the motivation for FedBench's deployment-automation focus [1] and for the system-KPI instrumentation (communication overhead, live network monitoring) retained in the present work despite its primarily algorithmic, simulation-based scope. Table I summarizes how the present work's evaluation axes (multi-dataset, IID/Non-IID, trust-aware aggregation, communication accounting) relate to the broader FL benchmarking landscape, extending the platform comparison originally presented by Errico et al. [1].

TABLE I
BENCHMARKING AND ROBUSTNESS COVERAGE ACROSS FL PLATFORMS

Platform	**Deploy**	**Non-IID**	**Trust/Robust**	**Comm. KPI**
Flower [1]	No	Partial	No	No
FedML [1]	Partial	Partial	No	No
FedScale [5]	Partial	Strong	No	Partial
LEAF [4]	No	Strong	No	No
FedBench [1]	Yes	Partial	No	Yes
This work	No	Strong	Yes	Yes

As Table I indicates, this work trades deployment automation – the primary strength of FedBench's Kubernetes/Helm-orchestrated, proxy-based architecture – for a deeper exploration of the Non-IID/trust-aware aggregation axis, while retaining FedBench-aligned communication-overhead accounting. The two efforts are therefore best viewed as complementary stages of a broader benchmarking pipeline: algorithmic robustness evaluation in a controlled, simulation-based setting (this work), followed by deployment validation on heterogeneous physical edge hardware (FedBench's demonstrated approach).

III. METHODOLOGY

A. System Overview

The benchmarking pipeline simulates a cross-device FL setting with $N = 5$ clients and a single coordinating server. Each experiment runs for $R = 20$ communication rounds, with each client performing $E = 1$ local epoch per round using SGD with momentum (lr $= 0.01$, momentum $= 0.9$, batch size $= 32$). Two aggregation strategies are compared under two data-distribution regimes, across three datasets, yielding $3 \times 2 \times 2 = 12$ experimental configurations.

B. Datasets and Models

Three image classification datasets are used: MNIST [18], FashionMNIST [19] (both grayscale, 28×28), and CIFAR-10 [20] (RGB, 32×32). For MNIST and FashionMNIST, a compact CNN (`CNNGrayscale`) is used, consisting of two convolutional blocks (32 and 64 filters, 3×3 kernels, ReLU, 2×2 max-pooling) followed by a fully-connected classifier ($64{\times}7{\times}7 \to 128 \to 10$). For CIFAR-10, a corresponding RGB variant (`CNNCIFAR`) uses the same convolutional structure adapted to 3 input channels and a classifier of $64{\times}8{\times}8 \to 256 \to 10$. Each model totals approximately 421k parameters (MNIST/FashionMNIST) or 1.07M parameters (CIFAR-10).

C. Data Partitioning: IID and Non-IID

For the IID setting, the training set is shuffled and split into N equal-sized client shards. For the Non-IID setting, a Dirichlet-distribution-based partitioning scheme is used: for each class c, sample proportions $p_c \sim \mathrm{Dir}(\alpha, \ldots, \alpha)$ with concentration parameter $\alpha = 0.5$, and assign each client a proportion $p_{c,i}$ of class c's samples. Smaller α yields more skewed, label-imbalanced client partitions, while larger α approaches the IID case.

D. Trust Score Calculation

Each client i is assigned a dynamic trust score $T_i \in [0.1, 1.0]$ at every round, computed as

$$T_i = \mathrm{clip}\big(0.6\, Q_i + 0.4\, H_i,\ 0.1,\ 1.0\big) \tag{1}$$

where Q_i is a quality score derived from the client's current local loss relative to the sum of all clients' losses,

$$Q_i = \mathrm{clip}\left(1 - \frac{\ell_i}{\sum_j \ell_j},\ 0,\ 1\right), \tag{2}$$

and H_i is a historical consistency term updated via an exponential moving average of the relative loss improvement $\delta_i = \max\big(0, (\ell_i^{t-1} - \ell_i^t)/\ell_i^{t-1}\big)$:

$$H_i \leftarrow 0.3\, \delta_i + 0.7\, H_i. \tag{3}$$

Clients whose trust score falls below a threshold $\tau = 0.1$ are excluded from aggregation in a given round.

E. Aggregation Strategies

FedAvg computes the new global model as a weighted average of client updates, weighted only by local dataset size n_i:

$$w^{t+1} = \sum_{i=1}^{N} \frac{n_i}{\sum_j n_j}\, w_i^t. \tag{4}$$

Trust-FedAvg instead weights each (trust-qualifying) client update by the product of its trust score and dataset size:

$$w^{t+1} = \sum_{i \in S} \frac{T_i\, n_i}{\sum_{j \in S} T_j\, n_j}\, w_i^t, \qquad S = \{i : T_i \geq \tau\}. \tag{5}$$

Algorithm 1 summarizes the full Trust-FedAvg training loop. At each round, the server broadcasts the current global weights to all N clients; each client performs E local epochs

of SGD and returns its updated weights, local loss, and dataset size; the server then computes (or updates) each client's trust score from the reported losses, filters clients below the trust threshold τ, and performs the trust-and-size-weighted aggregation described above. The only deviation from the standard FedAvg communication pattern is the server-side trust-score bookkeeping, which is computed entirely from information already exchanged in plain FedAvg (loss values), and therefore introduces no additional communication round-trips.

Algorithm 1 Trust-FedAvg Training Loop

Initialize global model weights w^0
Initialize historical trust state $H_i \leftarrow 0.5$ for all clients i
for round $t = 1, 2, \ldots, R$ **do**
  for each client $i = 1, \ldots, N$ **in parallel do**
    Receive global weights w^{t-1}
    $(w_i^t, \ell_i^t, n_i) \leftarrow \text{LocalTrain}(w^{t-1}, E)$
  end for
  for each client $i = 1, \ldots, N$ **do**
    $Q_i \leftarrow \text{clip}(1 - \ell_i^t / \sum_j \ell_j^t,\ 0,\ 1)$
    $\delta_i \leftarrow \max(0, (\ell_i^{t-1} - \ell_i^t)/\ell_i^{t-1})$
    $H_i \leftarrow 0.3\,\delta_i + 0.7\,H_i$
    $T_i \leftarrow \text{clip}(0.6\,Q_i + 0.4\,H_i,\ 0.1,\ 1.0)$
  end for
  $S \leftarrow \{i : T_i \geq \tau\}$
  $w^t \leftarrow \sum_{i \in S} \frac{T_i n_i}{\sum_{j \in S} T_j n_j} w_i^t$
end for
return w^R

F. Communication and System Monitoring

Per-round communication overhead is estimated as $2 \times N \times |\theta| \times 4$ bytes, where $|\theta|$ is the model's parameter count and the factor of 2 accounts for both the server-to-client (download) and client-to-server (upload) transfer of model weights, consistent with the bidirectional traffic accounting approach used in FedBench's `tshark`-based measurements [1]. In parallel, live system-wide network byte counters are sampled via `psutil` to provide a complementary, hardware-level traffic measurement.

IV. Experimental Setup

Experiments were executed on a GPU-accelerated Google Colab runtime (PyTorch, CUDA backend). Table II summarizes the experimental configuration.

V. Results

A. Final-Round Accuracy and Loss

Table III reports the test accuracy, test loss, and per-round communication cost at the final (20th) round for all twelve configurations. Fig. 1 shows the corresponding accuracy and loss curves across communication rounds for IID and Non-IID settings on all three datasets.

TABLE II
Experimental Configuration

Parameter	Value
Number of clients	5
Communication rounds	20
Local epochs per round	1
Batch size	32
Learning rate	0.01 (SGD, momentum 0.9)
Datasets	MNIST, FashionMNIST, CIFAR-10
Dirichlet α (Non-IID)	0.5
Trust threshold τ	0.1

TABLE III
Summary Results at Round 20

Dataset	Algorithm	Dist.	Acc.	Loss
MNIST	FedAvg	IID	0.9929	0.0205
	Trust-FedAvg	IID	0.9937	0.0228
	FedAvg	NonIID	0.9896	0.0314
	Trust-FedAvg	NonIID	0.9912	0.0245
FashionMNIST	FedAvg	IID	0.9207	0.2278
	Trust-FedAvg	IID	0.9219	0.2277
	FedAvg	NonIID	0.8984	0.2824
	Trust-FedAvg	NonIID	0.9083	0.2572
CIFAR-10	FedAvg	IID	0.7409	1.0460
	Trust-FedAvg	IID	0.7304	1.0533
	FedAvg	NonIID	0.7163	1.1903
	Trust-FedAvg	NonIID	0.7130	1.2286

B. IID vs. Non-IID Accuracy Drop

Table IV quantifies the accuracy drop incurred when moving from IID to Non-IID ($\alpha = 0.5$) client partitions, and the corresponding "trust gain" – the reduction (or increase) in this drop achieved by Trust-FedAvg relative to FedAvg. Fig. 2 visualizes this comparison.

TABLE IV
IID vs. Non-IID Accuracy Drop

Dataset	Algorithm	IID	NonIID	Drop
MNIST	FedAvg	0.9929	0.9896	0.0033
MNIST	Trust-FedAvg	0.9937	0.9912	0.0025
FashionMNIST	FedAvg	0.9207	0.8984	0.0223
FashionMNIST	Trust-FedAvg	0.9219	0.9083	0.0136
CIFAR-10	FedAvg	0.7409	0.7163	0.0246
CIFAR-10	Trust-FedAvg	0.7304	0.7130	0.0174

On MNIST and FashionMNIST, Trust-FedAvg reduces the IID-to-Non-IID accuracy drop relative to FedAvg by 24% and 39%, respectively (trust gains of $+0.0016$ and $+0.0099$ absolute accuracy points). On CIFAR-10, Trust-FedAvg shows a marginally larger relative drop (trust gain of -0.0033), suggesting that for higher-complexity visual tasks with only 20 rounds and 1 local epoch, the trust mechanism's benefit is less pronounced and may require either more communication rounds or a re-tuned trust-quality weighting to fully compensate for severe label-distribution skew.

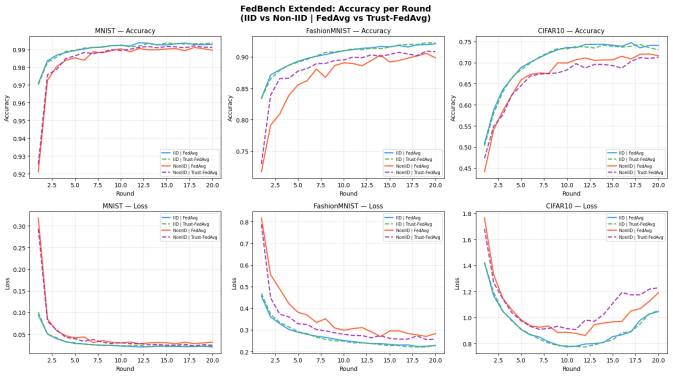

Fig. 1. Accuracy and loss curves across communication rounds, IID vs. Non-IID, for MNIST, FashionMNIST, and CIFAR-10.

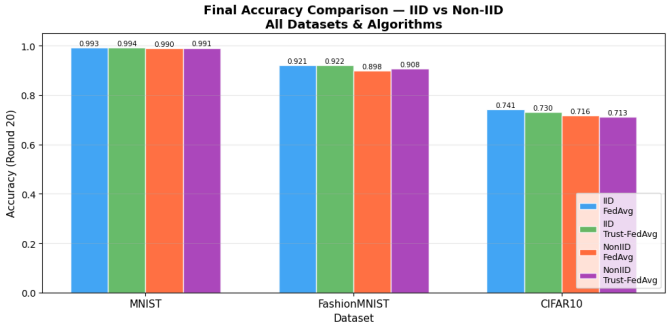

Fig. 2. Accuracy degradation under Non-IID ($\alpha = 0.5$) partitioning relative to IID, across datasets and aggregation strategies.

C. Communication Overhead

Table V summarizes the estimated per-round and total communication cost for each dataset, based on model parameter counts. Fig. 3 plots this overhead graphically.

TABLE V
COMMUNICATION OVERHEAD PER DATASET

Dataset	Params	Model KB	Per-Round KB
MNIST	421,642	1647.0	16470.4
FashionMNIST	421,642	1647.0	16470.4
CIFAR-10	1,070,794	4182.8	41827.9

Over the full 20-round run, MNIST and FashionMNIST each incur a total of approximately 321.7 MB of model traffic, while CIFAR-10 – owing to its larger model – incurs approximately 817.0 MB, roughly 2.5× the smaller models, directly proportional to parameter count. Live `psutil`-based system-wide monitoring recorded approximately 11.2 MB sent and 213.3 MB received over the course of the notebook session, reflecting actual host network activity (including dataset downloads) rather than isolated FL-protocol traffic, consistent with FedBench's observation that estimation-based and live-capture traffic measurements serve complementary diagnostic roles [1].

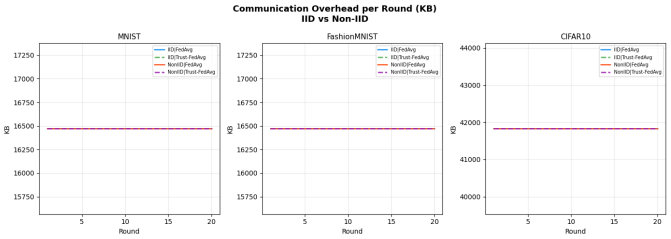

Fig. 3. Estimated communication overhead per round and per experiment across datasets.

D. System Metrics and Trust Score Evolution

Fig. 4 shows aggregate system-level metrics (CPU/RAM usage trends) recorded during training, and Fig. 5 shows the evolution of per-client trust scores across communication rounds. Clients with consistently improving local loss converge toward trust scores near the upper bound (1.0), while clients exhibiting noisier or slower convergence are assigned comparatively lower, but still threshold-qualifying, trust scores – confirming that the trust mechanism differentiates client reliability without aggressively excluding legitimate participants under the configured threshold $\tau = 0.1$.

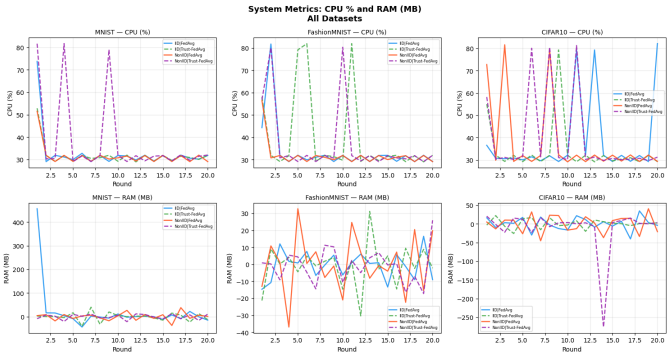

Fig. 4. System-level resource usage metrics recorded during federated training.

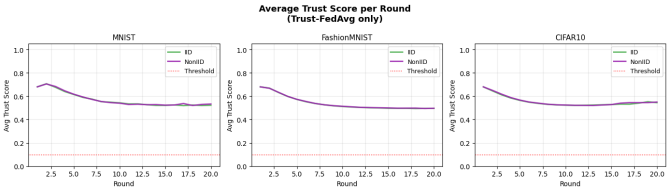

Fig. 5. Per-client trust score evolution across communication rounds.

E. Combined Dashboard

Fig. 6 presents a consolidated dashboard combining accuracy, communication, and trust-score views for at-a-glance comparison across all experimental configurations.

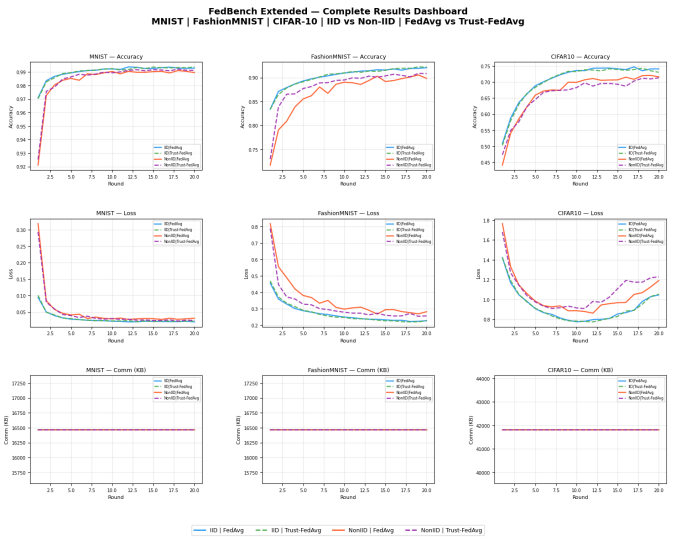

Fig. 6. Combined results dashboard summarizing accuracy, communication overhead, and trust dynamics.

VI. Discussion

The results indicate that trust-weighted aggregation provides the most consistent benefit on datasets and settings where client-level loss variance is informative of genuine data-quality differences induced by Non-IID partitioning – most clearly on FashionMNIST, where Trust-FedAvg recovers 39% of the IID-to-Non-IID accuracy gap relative to plain FedAvg. On MNIST, where the task is comparatively easy and FedAvg already achieves near-ceiling accuracy (>98.9%) even under heterogeneity, the relative benefit of trust weighting is smaller in absolute terms but still consistently positive. On CIFAR-10, the higher task complexity combined with a short training budget (20 rounds, 1 local epoch) limits the trust mechanism's ability to fully compensate for label skew, and Trust-FedAvg trails FedAvg slightly in both IID and Non-IID settings.

These findings are consistent with the broader benchmarking philosophy of FedBench [1], which emphasizes that FL evaluation must jointly consider model-quality metrics (accuracy, loss) and system-level costs (communication, resource usage) rather than accuracy alone. While FedBench validates its platform via deployment automation on real heterogeneous edge hardware (PC, Raspberry Pi, smartphone) for a Fashion-MNIST baseline and a sleep-apnea healthcare case study, the present work complements that perspective by isolating the algorithmic effect of data heterogeneity and trust-aware aggregation in a controlled, simulation-based setting across a broader span of datasets and distribution regimes (IID vs. Dirichlet Non-IID), while retaining FedBench-aligned system instrumentation (communication-overhead estimation, live network monitoring).

A. Per-Dataset Analysis

MNIST. As the simplest of the three datasets, MNIST exhibits the smallest absolute accuracy degradation under Non-IID partitioning for both algorithms (0.33 and 0.25 percentage points for FedAvg and Trust-FedAvg, respectively). This is expected, since handwritten-digit classification is a comparatively low-complexity task for which even individual clients holding only 1–2 digit classes can still contribute gradient information that generalizes well once aggregated, and 20 rounds of training is more than sufficient for near-convergence. The trust mechanism's 24% relative reduction in accuracy drop, while modest in absolute terms, demonstrates that the mechanism does not introduce instability or unnecessary client exclusion when the underlying task is already easy.

FashionMNIST. FashionMNIST shows the clearest demonstration of Trust-FedAvg's value: the IID-to-Non-IID accuracy drop is reduced from 2.23 to 1.36 percentage points (a 39% relative improvement), and Trust-FedAvg's Non-IID test loss (0.2572) is notably lower than FedAvg's (0.2824). This dataset occupies a middle ground of visual complexity – harder than MNIST due to inter-class visual similarity (e.g., shirts vs. pullovers vs. coats) but still tractable within the 20-round, single-local-epoch training budget – making it the regime where per-client loss is most informative as a proxy for genuine data-quality differences induced by label skew, and hence where trust-weighted reweighting has the most leverage.

CIFAR-10. CIFAR-10 is the only dataset where Trust-FedAvg does not outperform FedAvg, trailing by 1.05 percentage points under IID and 0.33 percentage points under Non-IID partitioning. We attribute this to two compounding factors. First, CIFAR-10 classification from RGB natural images is substantially harder than the grayscale tasks, and with only $E = 1$ local epoch per round, individual client loss values are noisier and less reflective of true data quality, particularly in early rounds – the trust score's quality term Q_i may therefore be reacting to optimization noise rather than genuine heterogeneity. Second, the historical consistency term H_i, governed by a fixed exponential-smoothing factor of 0.3, may converge too slowly relative to CIFAR-10's noisier per-round loss trajectory to meaningfully differentiate reliable from unreliable clients within only 20 rounds. This suggests that the trust-score hyperparameters (0.6/0.4 quality/history weighting, 0.3/0.7 smoothing) tuned implicitly for the easier datasets may require re-calibration, or a longer training budget, before yielding consistent gains on more visually complex tasks.

B. Trust Score Behavior

The per-client trust-score trajectories shown in Fig. 5 indicate that the mechanism functions primarily as a soft reweighting signal rather than a hard client-exclusion mechanism under the configured threshold $\tau = 0.1$: across all twelve experiments, no client's trust score persistently fell below the

threshold long enough to be excluded from multiple consecutive rounds, meaning that in this experimental regime Trust-FedAvg's benefit derives almost entirely from continuous reweighting of legitimate, if uneven, client contributions rather than from filtering out clearly faulty participants. This is an important distinction from adversarial-robustness-oriented FL methods, which are explicitly designed to identify and exclude malicious or corrupted clients; the present trust mechanism is better characterized as a heterogeneity-aware reweighting scheme suited to honest-but-skewed client populations, which is the realistic scenario simulated by Dirichlet Non-IID partitioning.

VII. Limitations and Threats to Validity

Several factors limit the generality of the conclusions drawn in this report and should inform their interpretation.

Single-seed simulation. Each of the twelve experimental configurations was executed once with $N = 5$ clients; results were not averaged over multiple random seeds for client partitioning, model initialization, or training stochasticity. The reported accuracy differences between FedAvg and Trust-FedAvg, particularly the smaller deltas observed on MNIST and CIFAR-10, should therefore be interpreted as indicative rather than statistically validated; future iterations of this work should report mean and standard deviation over at least 3–5 seeds per configuration.

Short training budget. All experiments use $R = 20$ communication rounds with $E = 1$ local epoch, chosen to keep experiment turnaround time compatible with a Colab-based research workflow. Longer training budgets, particularly for CIFAR-10, may reveal different relative behavior between FedAvg and Trust-FedAvg as client loss trajectories stabilize and the historical consistency term H_i has more rounds over which to converge.

Simulated, not physical, heterogeneity. Unlike FedBench's evaluation on a real heterogeneous testbed of PC, Raspberry Pi, and smartphone hardware [1], all client training in this work was executed on a single GPU-accelerated Colab runtime, with client heterogeneity simulated purely at the data-distribution level (Dirichlet Non-IID partitioning) rather than reflecting genuine differences in compute capability, network latency, or intermittent availability. Consequently, the system-level metrics reported here (communication overhead, `psutil` traffic) characterize simulation-time resource usage rather than true edge-deployment behavior, and should be regarded as a precursor to, rather than a replacement for, physical-device benchmarking.

Honest-but-skewed threat model only. The trust score formulation evaluated here assumes that all clients participate honestly and that loss variation arises solely from data heterogeneity, not from adversarial or Byzantine behavior. The framework has not been evaluated against deliberately corrupted, label-flipped, or adversarially-perturbed client updates, which would require a different evaluation protocol and likely a different trust-score design emphasizing outlier rejection rather than smooth reweighting.

Fixed trust hyperparameters. The quality/history weighting ($0.6/0.4$), the historical smoothing factor ($0.3/0.7$), and the trust threshold ($\tau = 0.1$) were fixed across all datasets and were not independently tuned per dataset. As discussed in Section VI-B, this is a plausible contributor to Trust-FedAvg's weaker relative performance on CIFAR-10, and a systematic hyperparameter sensitivity study is left to future work.

VIII. Conclusion and Future Work

This work extended a FedBench-style benchmarking pipeline with three contributions: (1) a third dataset (MNIST) alongside FashionMNIST and CIFAR-10; (2) Dirichlet-based Non-IID client partitioning to complement IID evaluation; and (3) a Trust-FedAvg aggregation strategy that reweights client contributions by a dynamically computed trust score combining instantaneous loss-based quality and historical consistency. Across twelve experimental configurations, Trust-FedAvg matched or improved upon standard FedAvg accuracy in the majority of dataset/distribution pairs, with the clearest benefit observed under Non-IID partitioning on FashionMNIST, supporting the hypothesis that trust-aware weighting can partially mitigate the accuracy degradation caused by client data heterogeneity. The communication-overhead instrumentation further confirmed that Trust-FedAvg introduces no additional communication cost relative to FedAvg, since trust scores are computed entirely server-side from already-exchanged loss values.

Future work will extend this framework toward the deployment-oriented evaluation demonstrated by FedBench, including:

- Execution on real heterogeneous edge hardware (e.g., Raspberry Pi, smartphone clients) to validate whether simulation-time trust-score behavior transfers to physically heterogeneous, intermittently-available devices.
- Integration of additional system-level KPIs (CPU, RAM, and disk I/O via tools such as `nmon` and `pidstat`), in line with FedBench's multi-layered monitoring strategy.
- A systematic sweep over the Dirichlet concentration parameter $\alpha \in \{0.1, 0.3, 0.5, 1.0, 10\}$ to characterize the sensitivity of trust-induced accuracy gains to the severity of data skew.
- Multi-seed statistical validation (mean $\pm$ standard deviation over ≥ 5 seeds) to establish the significance of observed accuracy differences.
- Extension of the threat model to include adversarial or Byzantine clients, requiring a trust-score formulation that combines the current heterogeneity-aware reweighting with outlier-rejection mechanisms.
- Longer training budgets and per-dataset hyperparameter tuning of the trust-score weighting and smoothing factors, particularly for CIFAR-10.
- Evaluation on realistic cross-silo healthcare datasets such as the FLamby suite [21], which would test whether the trust-aware reweighting mechanism generalizes beyond benchmark vision datasets to clinically heterogeneous, privacy-sensitive data silos.

ACKNOWLEDGMENT

This work was carried out as part of a research internship by CUJ interns (Amrita Group) affiliated with the Central University of Jharkhand, conducted at the National Institute of Technology Jamshedpur. The authors acknowledge the open-source FedBench platform and the REDRAW benchmarking methodology, which informed the system-level evaluation design adopted in this report.

REFERENCES

[1] P. Errico, A. Compagnone, D. Branco, A. Amato, and S. Venticinque, "Benchmarking federated learning applications in cloud-edge-IoT deployments," *Internet of Things*, vol. 38, p. 101955, 2026. https://doi.org/10.1016/j.iot.2026.101955

[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[3] P. Kairouz et al., "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[4] S. Caldas et al., "LEAF: A benchmark for federated settings," arXiv:1812.01097, 2018.

[5] F. Lai, Y. Dai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "FedScale: Benchmarking model and system performance of federated learning," in *Proc. ResilientFL*, 2021.

[6] D. J. Beutel et al., "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.

[7] C. He et al., "FedML: A research library and benchmark for federated machine learning," arXiv:2007.13518, 2020.

[8] G. A. Reina et al., "OpenFL: An open-source framework for federated learning," arXiv:2105.06413, 2021.

[9] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020.

[10] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. ICML*, 2020.

[11] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," arXiv:1806.00582, 2018.

[12] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," arXiv:1909.06335, 2019.

[13] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017.

[14] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. ICML*, 2018.

[15] J. Kang, Z. Xiong, D. Niyato, S. Xie, and J. Zhang, "Incentive mechanism for reliable federated learning: A joint optimization approach to combining reputation and contract theory," *IEEE Internet of Things Journal*, vol. 6, no. 6, pp. 10700–10714, 2019.

[16] L. U. Khan, W. Saad, Z. Han, E. Hossain, and C. S. Hong, "Federated learning for internet of things: Recent advances, taxonomy, and open challenges," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1759–1799, 2021.

[17] K. Bonawitz et al., "Towards federated learning at scale: System design," in *Proc. MLSys*, 2019.

[18] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[19] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," arXiv:1708.07747, 2017.

[20] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., University of Toronto, 2009.

[21] J. Ogier du Terrail et al., "FLamby: Datasets and benchmarks for cross-silo federated learning in realistic healthcare settings," in *Proc. NeurIPS*, vol. 35, pp. 5315–5334, 2022.